

CICLO DE COMPLEMENTACIÓN

LICENCIATURA EN CIENCIA DE DATOS

CLCD 182 - SEMINARIO DE
ACTUALIZACIÓN I

Clases

Semana 04





Temas a Desarrollar

- Funciones
- Parámetros de funciones
- Anotaciones de funciones
- Funciones lambda
- Recursividad
- Decoradores
- Corrutinas
- Caching de funciones
- Programación funcional



Funciones

Las funciones nos permiten realizar operaciones específicas con entradas para producir una salida.

Tipos

- Nativas
- Definidas por el usuario
- Librerías o módulos

Por son importante?

- **Reusabilidad:** Escribe el código una vez, úsalo muchas veces. ¡Evita la repetición!
- **Modularidad:** Divide tu código en partes más pequeñas y manejables. ¡Código más fácil de leer y mantener!

Definición

```
def nombre_función(argumentos_o_parámetros):  
    # código  
    return retorno
```



Funciones

Como se pueden pasar los argumentos?

```
def por_defecto(valor_1:int, valor_2:int=0):  
    pass  
  
por_defecto(argumento_1)
```

```
def por_posicion(valor_1, valor_2):  
    pass  
  
por_posicion(argumento_1, argumento_2)
```

```
def por_nombre(valor_1, valor_2):  
    pass  
  
por_nombre(valor_2=argumento_1, valor_1=argumento_2)
```



Parámetros de funciones

Cual es el concepto del pasaje de los parámetros?

- **Por valor:** creará una copia local de la variable, tipos inmutables.
 - a. int
 - b. float
 - c. str
 - d. bool
- **Por referencia:** se actuará directamente sobre la variable pasada, tipos mutables.
 - a. list
 - b. set
 - c. dict



Parámetros de funciones

El argumento ***args** nos permite recibir un número de argumentos indefinidos. En este caso lo recibiremos como una tupla.

```
def suma_args(*numeros:int|float) -> int|float:
    total:int|float = 0
    for n in numeros:
        total += n
    return total

resultado:int|float = suma_args(1, 3, 5, 4)
resultado:int|float = suma_args(6, 4, 10, 20, 4, 6, 7)
```



Parámetros de funciones

El argumento ****kwargs** nos permite recibir un número de argumentos indefinidos pero con su nombre y valor. En este caso lo recibiremos como un diccionario.

```
def suma_kwargs(**kwargs) -> int|float:
    suma:int|float = 0;
    for key, value in kwargs.items():
        suma += value
    return suma

resultado:int|float = suma_kwargs(argumento_1=5, argumento_2=20, argumento_3=23)
```



Anotaciones de funciones

Motivación:

- Evitar errores en el pasaje de argumentos por el duck typing.
- Describe el funcionamiento.
- Describe exactamente qué tipos de datos recibe.
- Describe exactamente qué tipo/s de dato/s retorna.

```
def multiplica_por_3(numero: int) -> int:
    """ Función que multiplica un número entero por 3

    Args:
        numero (int): Número entero a multiplicar

    Returns:
        (int): El número multiplicado por 3
    """
    return numero*3
```

Podemos realizar un chequeo estático de tipos con **mypy**.



Funciones lambda

Son funciones sin nombre y tiene la particularidad que solo pueden tener una línea de código.

Definición:

```
lambda a, b : a + b
```

Donde utilizarla?

- En argumentos de funciones.
- En algún caso particular de asignación de valores.

Donde no utilizarla?

- En la asignación de una variable.



Recursividad

Es la división de un problema en tareas o subtareas que se resuelven de la misma forma o con el mismo código fuente.

Secciones:

- La función se llama a sí misma.
- Valor de retorno sin volver a llamarse. Es muy importante porque de lo contrario, la función se llamaría de manera indefinida.

```
def factorial_normal(n:int) -> int:
    if n == 0: return 1
    resultado:int = 1
    for i in range(1, n+1):
        resultado *= i
    return resultado
```

```
def factorial_recursivo(n:int) -> int:
    if n == 0: return 1
    return n * factorial_recursivo(n-1)
```



Decoradores

Modifican el comportamiento de una función y se utilizan el @<decorador>.

Tipos:

- Nativos de python.
 - a. @staticmethod
 - b. @classmethod
 - c. @property
- Desarrollados por el usuario.
 - a. Veamos un ejemplo



Decoradores

```
def mi_decorador(funcion):
    def nueva_funcion(a:int|float, b:int|float) -> int|float:
        print("Se va a llamar")
        c = funcion(a, b)
        print("Se ha llamado")
        return c
    return nueva_funcion

@mi_decorador
def suma(a:int|float, b:int|float) -> int|float:
    print("Entra en funcion suma")
    return a + b

print("El resultado de sumar 5 + 8 es: ", suma(a=5, b=8))
```

A horizontal bar with a teal segment on the left and an orange segment on the right.

Generadores

Son generadores de datos. Para esto en lugar de utilizar **return** se utiliza **yield**.

Utilizan menos recursos que otros generadores (**range**).

Podemos crear clases con la sobrecarga de **__iter__** y **__next__**.

Ejemplo:



Generadores

Son generadores de datos. Para esto en lugar de utilizar **return** se utiliza **yield**.

Utilizan menos recursos que otros generadores (**range**).

Podemos crear clases con la sobrecarga de **__iter__** y **__next__**.

Ejemplo:

```
def generador():  
    n = 1  
    yield n  
  
    n += 1  
    yield n  
  
    n += 1  
    yield n
```



Generadores

Son generadores de datos. Para esto en lugar de utilizar **return** se utiliza **yield**.

Utilizan menos recursos que otros generadores (**range**).

Podemos crear clases con la sobrecarga de **__iter__** y **__next__**.

```
for i in generador():  
    print('El resultado es: ', i)
```

```
def generador():  
    n = 1  
    yield n  
  
    n += 1  
    yield n  
  
    n += 1  
    yield n
```



Generadores

Son generadores de datos. Para esto en lugar de utilizar **return** se utiliza **yield**.

Utilizan menos recursos que otros generadores (**range**).

Podemos crear clases con la sobrecarga de **__iter__** y **__next__**.

```
for i in generador():  
    print('El resultado es: ', i)
```

```
def generador():  
    n = 1  
    yield n  
  
    n += 1  
    yield n  
  
    n += 1  
    yield n
```

```
g = generador()  
print('El resultado de invocar al generador y pasarselo a next es: ', next(g))  
print('El resultado de invocar al generador y pasarselo a next es: ', next(g))  
print('El resultado de invocar al generador y pasarselo a next es: ', next(g))
```



Corrutinas



Son consumidores de datos. Utilizan **yield** en lugar de **return**.

El código solo se ejecuta cuando se lo solicita.

Ejemplo:



Corrutinas

Son consumidores de datos. Utilizan **yield** en lugar de **return**.

El código solo se ejecuta cuando se lo solicita.

Ejemplo:

```
def encontrar(patron):  
    """ Busca un patrón en la entrada.  
    """  
    print("Buscando", patron)  
    while True:  
        linea = (yield) # no es un retorno, sino un ingreso de datos|  
        if patron in linea:  
            print(f'Se encontró el patrón {patron} en la línea {linea}')
```



Corrutinas

Son consumidores de datos. Utilizan

yield en lugar de **return**.

El código solo se ejecuta cuando se lo solicita.

Ejemplo:

```
def encontrar(patron):  
    """ Busca un patrón en la entrada.  
    """  
    print("Buscando", patron)  
    while True:  
        linea = (yield) # no es un retorno, sino un ingreso de datos  
        if patron in linea:  
            print(f'Se encontró el patrón {patron} en la línea {linea}')
```

```
buscar = encontrar('corrutina')  
next(buscar)  
buscar.send("Prueba línea 1")  
buscar.send("Esto es una función")  
buscar.send("Estoy aprendiendo corrutinas")  
buscar.send("Esto es perfecto!")  
buscar.close()
```



Caching de funciones

Almacenamiento de datos en memoria para futuras ejecuciones.

Existen por lo tanto dos posibilidades:

- **cache miss:** Si ejecutamos la función y el resultado no existe, entonces se calcula y se almacena.
- **cache hit:** Si ejecutamos la función y el resultado ya fue calculado.

Algunas cosas a tener en cuenta:

- Es útil cuando trabajamos con funciones donde el cálculo es intenso.
- Es necesario conocer la distribución de los argumentos de la función. Si los valores son muy dispares, no tendremos **cache hit**.
- Mejorar el tiempo de respuesta, pero se paga con el incremento del uso de memoria.



Caching de funciones con diccionarios

```
def es_primo(num:int) -> bool:
    for n in range(2, num):
        if num % n == 0:
            return False
    return True
```

```
def es_primo_con_cache(num:int, _cache:dict[int, bool]={}) -> bool:
    if num not in _cache:
        _cache[num] = True
        for n in range(2, num):
            if num % n == 0:
                _cache[num] = False
                break
    return _cache[num]
```

Caching de funciones con @lru_cache (nativo)



```
def es_primo(num:int) -> bool:
    for n in range(2, num):
        if num % n == 0:
            return False
    return True
```

```
from functools import lru_cache

@lru_cache(maxsize=32)
def es_primo_con_cache_lru(num:int) -> bool:
    for n in range(2, num):
        if num % n == 0:
            return False
    return True
```



Programación funcional

Se basa principalmente en el uso de funciones y no existen bucles (for o while)

La función **map** toma dos entradas:

- Una lista o iterable.
- Una función, que será aplicada a cada uno de los elementos.
- Retorna una lista con el resultado de aplicar la función a cada elemento de la lista.

```
lista = [1, 2, 3, 4, 5]
lista_pordos = map(lambda x: 2*x, lista)

print('Los valores de la lista multiplicados por 2 es: ', list(lista_pordos))
```

```
Los valores de la lista multiplicados por 2 es: [2, 4, 6, 8, 10]
```



Programación funcional

La función **filter** toma dos entradas:

- Una lista o iterable.
- Una función, que será aplicada a cada uno de los elementos.
- Retorna una lista con los datos filtrados.

```
lista = [7, 4, 16, 3, 8]
pares = filter(lambda x: x % 2 == 0, lista)

print('Los valores pares son: ', list(pares))

Los valores pares son: [4, 16, 8]
```



Programación funcional

La función **reduce** toma dos entradas:

- Una lista o iterable.
- Una función, que será aplicada a cada uno de los elementos.
- Retorna un valor como resultado final de aplicar dicha función.
- El acumulador (**acc**) es el valor devuelto en la iteración anterior, que va acumulando un resultado hasta que llegamos al final.
- El valor (**val**) es cada uno de los elementos de nuestra lista, que en nuestro caso vamos añadiendo al acumulador.

```
from functools import reduce
lista = [1, 2, 3, 4, 5]
suma = reduce(lambda acc, val: acc + val, lista)
print('La suma de los valores es: ', suma)
```

```
La suma de los valores es: 15
```